Roadmap Técnico & Caderno de Decisões de Engenharia

Sistema Inteligente de Gestão de Ativos (MVP 1)

1. Visão Arquitetural e Princípios de Engenharia

O presente roadmap técnico define as diretrizes de desenvolvimento, escolhas de stack tecnológica, análise de riscos e arquitetura do MVP 1 do **Sistema Inteligente de Gestão de Ativos**.

1.1 Metas e Restrições Críticas de Engenharia

- **Operação 100% Offline:** A inferência da inteligência artificial e a persistência dos dados devem ocorrer localmente no dispositivo móvel, sem tolerância a falhas por falta de conectividade em galpões e eventos remotos.
- **Escopo Estrito de Classes:** Detecção e contagem agrupada de **Mesa Bistrô Dobrável** e **Gradil**. Não contempla rastreamento individual (RFID/Serial) nem geolocalização dos ativos.
- **Tolerância Máxima de Erro:** O erro médio absoluto de contagem (MAPE) fornecido pelo modelo deve ser inferior ou igual a **10,0%** em relação ao gabarito real nas condições de validação homologadas.
- **Prazo Final Imutável:** Entrega e homologação do MVP até o dia **11 de Dezembro de 2026**.

1.2 Princípio da Engenharia Concorrente

Para cumprir o prazo de 11 de dezembro, a engenharia de software e a engenharia de dados/IA operam de forma desacoplada e concorrente:

- O aplicativo móvel não depende da versão final da rede neural para iniciar seu desenvolvimento.
- Na Sprint 2, o app mobile utiliza uma inferência *mock* baseada em um modelo preliminar de teste gerado na Sprint 1 (ou payload estruturado em JSON com coordenadas simuladas) para construir a captura de câmera, decodificação em buffer, renderizador de *bounding boxes* em tela e banco local SQLite.
- Em paralelo, a frente de IA finaliza a anotação do dataset completo, treina e quantiza o modelo definitivo, realizando a substituição transparente do arquivo de modelo no aplicativo móvel antes da bateria de testes da Sprint 3.

2. Cronograma Macro e Linha do Tempo das Sprints

O cronograma operacional organiza o esforço total de 81 Story Points em três sprints e uma fase de sustentação:[Sprint 1: Concluída] ——▶ [Sprint 2: 08/Out a 05/Nov] ——▶ [Sprint 3: 06/Nov a 11/Dez] ——▶ [Pós-MVP]

(18 SP - Fundação) (39 SP - Core IA + Mobile) (24 SP - UX, Dados & Validação) (8 SP - MLOps)

Fase / Sprint	Período	Capacidade	Foco Técnico Principal
Sprint 1 (Concluída)	01/Set – 07/Out	18 SP	Fundação & Prova de Conceito
Sprint 2 (Em Andamento)	08/Out – 05/Nov	39 SP	Core IA & Motor de Inferência Offline
Sprint 3 (Planejada)	06/Nov – 11/Dez	24 SP	Conferência, Persistência & Ace Final
Pós-MVP (Sustentação)	Pós 11/Dez	8 SP	MLOps & Retreinamento Contínu

3. Caderno de Decisões Técnicas por Componente do Backlog

Abaixo estão destrinchadas as discussões técnicas não óbvias, alternativas avaliadas e recomendações de engenharia para cada elemento do produto:

3.1 Framework e Linguagem do Aplicativo Mobile

- **Contexto Atual:** A Sprint 1 entregou uma estrutura básica com página estática em HTML sem funções.
- **Pergunta Técnica Não Óbvia:** Como migrar do protótipo estático em HTML para um aplicativo com acesso nativo de baixa latência à câmera e renderização acelerada por CPU de dezenas de caixas delimitadoras, sem reconstruir todo o frontend do zero?
- **Alternativas Analisadas:**
    a. *WebView Pura / Cordova*: Baixo custo de reaproveitamento do HTML, porém sofre com alto consumo de memória, travamento no processamento de imagens e latência intolerável na manipulação de tensores no JavaScript.
    b. *Capacitor com Plugin Nativo*: Reaproveita o HTML existente para a interface do usuário, enquanto a inferência e manipulação de imagem ocorrem em thread nativa em background (Kotlin/Java no Android).
    c. React Native: Desenvolvimento cross-platform nativo com acesso direto às APIs de câmera e renderização acelerada pela CPU nativa em ambos Android e iOS.
- Solução Proposta: React Native como escolha primária de engenharia, permitindo acesso nativo de baixa latência à câmera via plugins React Native e renderização acelerada de dezenas de caixas delimitadoras. A integração do modelo TFLite é realizada através de bibliotecas nativas para ambos os plataformas Android (Kotlin/Java) e iOS (Swift).

3.2 Arquitetura do Modelo de Visão Computacional

- **Pergunta Técnica Não Óbvia:** Qual arquitetura de rede neural detector garante alto poder de discriminação em objetos densamente agrupados e com padrão repetitivo (como barras de gradis sobrepostos), respeitando o teto térmico e de memória de smartphones intermediários?
- **Alternativas Analisadas:**
    a. *Faster R-CNN (Two-stage)*: Acurácia superior em objetos pequenos, porém inviável para execução local em celulares comuns (latência > 8s por imagem, modelo > 150MB).
    b. *SSD MobileNet v2*: Muito leve, mas historicamente deficiente na detecção de estruturas vazadas e caixas muito próximas.
    c. *YOLOv8 Nano (YOLOv8n) ou YOLO26n*: Arquitetura single-stage sem âncoras (anchor-free), com ~3.2 milhões de parâmetros e excelente equilíbrio entre localização e tempo de resposta.
- **Solução Proposta: YOLO26n**. A solução será integrada diretamente ao fluxo de trabalho do Roboflow, utilizando seus checkpoints para carregamento e ajustando os pesos pré-treinados a partir do dataset COCO via transfer learning.

3.3 Runtime de Inferência 100% Offline no Dispositivo Móvel

- **Pergunta Técnica Não Óbvia:** Como empacotar e executar o grafo neural de forma agnóstica de hardware, alcançando tempo de inferência inferior a 1,5 segundos em CPUs móveis de baixo custo sem depender de internet?
- **Alternativas Analisadas:**
    a. *PyTorch Mobile*: Complexo de configurar e binário do runtime muito pesado (> 40MB apenas de bibliotecas base).
    b. *ONNX Runtime Mobile*: Excelente portabilidade, suporte a quantização INT8, mas documentação e wrappers em Flutter menos maduros.
    c. *TensorFlow Lite (TFLite)*: Padrão consolidado para plataformas móveis, com suporte nativo à aceleração XNNPACK em CPU ARM e delegates de GPU/NPU (NNAPI).
- **Solução Proposta:** Exportar o modelo para **TFLite com quantização de pós-treinamento FP16 (Float16)**. O arquivo final `.tflite` terá tamanho aproximado de 6 a 9 MB. O runtime móvel executará a inferência utilizando 4 threads de CPU via *delegate* XNNPACK, garantindo tempo de processamento entre 600ms e 1200ms em smartphones padrão (ex.: Snapdragon 680 ou similar).

3.4 Gestão de Memória da Câmera e Prevenção de Out-Of-Memory (OOM)

- **Pergunta Técnica Não Óbvia:** Como manipular fotos capturadas em resoluções modernas de 12MP a 48MP (gerando bitmaps descompactados de 40MB a 150MB) sem causar estouro de memória e crash do aplicativo durante a preparação do tensor de entrada?
- **Solução Proposta:** A imagem capturada pela câmera nativa não deve ser carregada integralmente na memória heap do Flutter/JavaScript. A decodificação do arquivo JPEG deve ser realizada diretamente na camada nativa (C++/Java) com redimensionamento proporcional e *letterboxing* direto para a dimensão de entrada da rede (**640×640 pixels em RGB normalizado [0.0, 1.0]**). O bitmap em alta resolução é salvo compactado diretamente em disco no cache temporário, enquanto apenas a matriz reduzida de `[1, 3, 640, 640]` float32 é mantida em memória para a inferência.

3.5 Tratamento de Oclusões e Algoritmo NMS para Gradis e Mesas Empilhadas

- **Pergunta Técnica Não Óbvia:** Quando gradis são enfileirados ou mesas bistrô são empilhadas, as caixas delimitadoras dos objetos adjacentes apresentam altíssima sobreposição. Como evitar que o algoritmo NMS (Non-Maximum Suppression) tradicional elimine objetos legítimos considerando-os falsas duplicatas?
- **Solução Proposta:**
    a. **Calibração de Limiares Diferenciados:** Em vez de usar um limiar de IoU (Intersection over Union) global, adotar IoU = 0.50 para Mesa Bistrô e elevar o

limiar para IoU = 0.60 com *Soft-NMS* para Gradis, permitindo que caixas adjacentes muito próximas não sejam sumariamente suprimidas.

b. **Protocolo Operacional de Captura Angular:** Orientar o conferente através de um guia visual na tela de câmera para capturar as pilhas de gradil em ângulo diagonal de 30° a 45° em relação ao plano frontal. Esse ângulo expõe as arestas superiores e laterais de cada gradil, viabilizando a separação visual inequívoca das peças.

3.6 Interface de Conferência Ergonômica em Ambiente Hostil

- **Pergunta Técnica Não Óbvia:** Em galpões com poeira, baixa luminosidade ou uso de luvas de proteção, como viabilizar a conferência e correção ágil das contagens sem exigir manipulação de gestos de alta precisão na tela do celular?
- **Solução Proposta:** Arquitetura de interface em duas camadas:
  a. *Ajuste Numérico Imediato (Camada Macro):* Exibição de um placar de contagem fixo no topo com botões grandes de incremento e decremento (+ e -) para cada classe. Se a IA contou 48 gradis e o operador sabe que o palete contém 50, ele faz o ajuste em 2 toques rápidos sem precisar interagir com as caixas.
  b. *Correção Visual por Ponto (Camada Micro):* A imagem capturada permite pinça para zoom (*pinch-to-zoom*). O operador pode tocar em uma caixa verde existente para apagá-la (falso positivo) ou tocar em um objeto não detectado para inserir uma marcação (falso negativo).

3.7 Persistência de Dados e Auditoria Local Offline

- **Pergunta Técnica Não Óbvia:** Como persistir inventários, lotes e imagens comprovatórias de forma confiável e com integridade referencial sem banco de dados em nuvem?
- Solução Proposta: Utilizar SQLite embarcado (via Room no Android ou bibliotecas nativas do React Native como react-native-sqlite-storage). O banco armazena apenas dados estruturados: ID do inventário, data/hora, identificador do usuário, total por classe, tempo de inferência, status de confirmação e as coordenadas JSON de todas as caixas validadas. A foto original e a foto com as anotações visuais são salvas compactadas em JPEG (85% de qualidade) no armazenamento privado do app (app_data/inventarios/), registrando apenas o caminho do arquivo no banco.

3.8 Controle de Acesso e Perfis sem Internet (RBAC Offline)

- **Pergunta Técnica Não Óbvia:** Como garantir que terceirizados e funcionários comuns não alterem configurações, não excluam históricos de conferência e assinem suas contagens de forma rastreável sem depender de autenticação OAuth ou APIs na nuvem?

- **Solução Proposta:** Autenticação local robusta baseada em PIN numérico de 4 a 6 dígitos ou senha com derivação de chave criptográfica usando **PBKDF2 com Salt** armazenado no SQLite. A chave mestre é protegida pelo Android KeyStore / iOS Keychain. São configurados três papéis:
    - *Dono / Administrador:* Acesso irrestrito a configurações de limiar de confiança, exclusão de histórico e cadastro de novos usuários.
    - *Funcionário:* Acesso à captura, conferência, correção e visualização do histórico dos últimos 30 dias (sem permissão de exclusão).
    - *Terceirizado:* Acesso estrito à tela de captura, contagem e confirmação da carga atual. Não acessa histórico anterior nem menus administrativos.

3.9 Protocolo Experimental de Validação da Meta de Erro <= 10%

- **Pergunta Técnica Não Óbvia:** Como comprovar formalmente perante os critérios de aceite que a solução atende à exigência contratual de erro máximo de 10% de forma estatisticamente válida?
- **Solução Proposta:** O teste de homologação na Sprint 3 seguirá um protocolo de validação cego:
    - Coleta de um lote exclusivo de **50 imagens de teste** em ambientes de clientes parceiros, cobrindo 4 estratos obrigatórios: (1) Galpão coberto com baixa luminosidade; (2) Pátio aberto com luz solar intensa; (3) Cargas densamente agrupadas (lotes de 20 a 100 itens); (4) Peças com oclusão parcial de até 40%.
    - Todas as 50 fotos terão sua contagem real (*Ground Truth*) verificada e assinada manualmente por dois auditores.
    - O app executará a inferência sobre as fotos em modo avião. O erro será medido pelo MAPE (*Mean Absolute Percentage Error*):

MAPE = (1 / N) * Σ (| Contagem IA - Contagem Real | / Contagem Real) * 100%

- O critério de aprovação exige que o MAPE seja inferior a **10,0%** para cada uma das classes de forma independente.

3.10 Coleta Incremental e MLOps Pós-MVP

- **Pergunta Técnica Não Óbvia:** Como preparar a arquitetura do MVP para evoluir continuamente com os dados corrigidos pelos conferentes em campo sem penalizar o armazenamento do smartphone?
- **Solução Proposta:** Implementar uma rotina de descarte inteligente: imagens em que a contagem da IA foi aceita sem nenhuma alteração manual são descartadas após 15 dias. Já as fotos em que o operador realizou correções manuais superiores a 10% no total de itens são marcadas como *'Discrepância de Treino'*. Quando o dispositivo detectar conexão a uma rede Wi-Fi e bateria superior a 50%, um serviço em segundo plano empacota essas fotos com as coordenadas corrigidas e envia para um bucket em

nuvem (AWS S3 ou Google Cloud Storage) para retraining supervisionado no ciclo Pós-MVP.

4. Matriz de Riscos Técnicos e Planos de Mitigação

Risco Técnico Identificado	Severidade	Probabilidade
1. Oclusão severa em gradis reduzir recall	Alta	Média
2. Celulares de entrada esquentarem ou sofrerem lentidão	Alta	Baixa
3. Falha de foco da câmera em ambientes escuros	Média	Alta
4. Tempo de anotação das 150 fotos restantes estourar o cronograma	Alta	Baixa

5. Plano de Ação Imediato para a Sprint 2

1. **Dia 08/Out a 15/Out:** Finalizar anotação assistida das 150 imagens restantes no Roboflow/CVAT; congelar dataset com 200 imagens balanceadas e divididas em 70% treino, 15% validação e 15% teste.
2. **Dia 16/Out a 25/Out:** Executar scripts de treinamento YOLO26n com data augmentation; validar mAP@50 > 0.85; converter artefato para TFLite FP16.
3. Dia 18/Out a 28/Out (Paralelo Mobile): Estruturar projeto base em React Native; integrar plugin nativo de câmera com controle de foco e redimensionamento para 640×640.
4. **Dia 29/Out a 05/Nov:** Embarcar modelo TFLite no app móvel; testar ciclo completo de captura de foto, inferência offline e exibição das caixas delimitadoras coloridas na tela em modo avião.